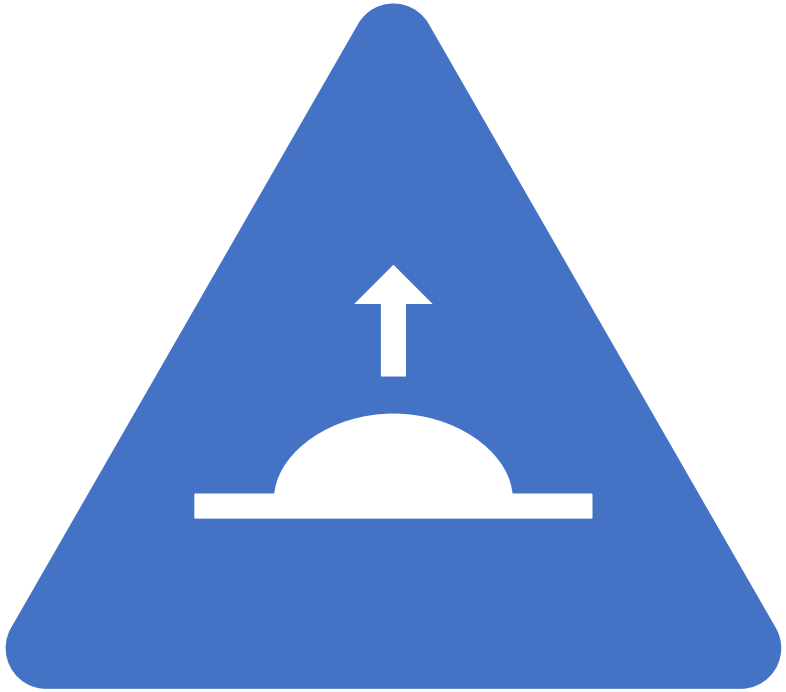
Unit 2: Storage Class, Predefined Processor, Types of Error



By: Rishi Kumar

Storage Classes

- Variables in C can have not only *data type* but also *storage class* that provides information about their location and visibility. The storage class decides the portion of the program within which the variables are recognized.

- The storage class is another qualifier (like **long** or **unsigned)** that can be added to a variable declaration as shown below:

```
auto int count;
register char ch;
static int x;
extern long total;
```

Storage Classes

- Storage class are used to represents following information about variable:
  - scope of a variable.
  - initial value of variable
  - where the variable will be stored
  - Lifetime of the variable
    - Automatic(auto)
    - External(extern)
    - Register
    - static

Storage Classes

Storage class	Storage	Default value	Scope	Lifetime
auto	memory	Garbage	Local , inside the block where it is defined	Till the control remains within the block in which the variable is defined.
static	Memory	Zero	Local , inside the block where it is defined	Value of the variable persists between different function calls.
register	CPU register	Garbage	Local , inside the block where it is defined	Till the control remains within the block in which the variable is defined.
extern	memory	Zero	global	as long as the program is running

auto vs. static Storage Class

```
int main()
{
  auto int a,b;
//auto storage class variable
  printf("\n %d %d", a,b);
}
```

If auto variable will not be initialized, the output will be any garbage value.

```
int main()
{
  static int a,b;
//static storage class variable
  printf("\n %d %d", a,b);
}
```

If static variable will not be initialized, the output will be 0.

The static variable is a variable that is capable of retaining its value between multiple numbers of function calls in a program.

auto vs. static Storage Class

```
int main()
{
   show();
   show();
   show();
   return 0;
}
void show()
{
   int a=10;  //auto variable
   a++;
   printf("\n %d",a);
}
```

O/P:11
11
11

```
int main()
{
   show();
   show();
   show();
   return 0;
}
void show()
{
   static int a=10;   //static variable
   a++;
   printf("\n %d",a);
}
```

O/P: 11
12
13

Storage Classes

static variables	auto variables
remains in memory while the program is running.	destroyed when a function call where the variable was declared is over.
Preserve their value even after they are out of their scope.	Do not preserve their value even after they are out of their scope.
Default value is zero.	Default value is garbage value.

Extern/global variable Storage Class

```
int a=10;    //extern or global variable,
can be accessed by all
                the functions
int main()
{
   fun1();
   fun2();
   fun2();
   fun1();
   return 0;
}
void fun1()
{
   a--;
   printf("\n %d",a);
   }
void fun2()
{
   a++;
   printf("\n %d",a);
}
```

O/P: 9

10

11

10

Pre-Defined Processor

Predefined Processor

- File Inclusion
- Macro
- Removing Comments
- Conditional Compilation

Command Line Argument

Predefined Processor

- C preprocessor is not a part of the compiler but is a separate step in the compilation process.
- All preprocessor directives begin with a hash symbol (#).
- A preprocessor is a system software.
- It performs preprocessing of the High-Level Language.
- Preprocessing is the first step of the language processing system.

Preprocessor Tasks

1. File inclusion

- Including all the files from library into the program.

For example, #include<stdio.h>

- the file will be searched in the standard directory.

- Preprocessor will include all the contents of library file stdio.h in the program.

#include “stdio.h”

- File will be searched in the current source directory.

Preprocessor Tasks

2. Macro Expansion

- A macro is a fragment of code which has been given a name.
- defined by using # define preprocessor directive.
- Before the source code passes to the compiler, C preprocessor examines for macro definitions.
- If it finds **#define** directive, then it search for the macro templates in the program and replaces the macro template with the appropriate macro expansion.
- After this process, program handed over to the compiler.

Preprocessor Tasks

- **object like macro**

  #define max 100

- **function like macro**
  - The macros can take function like arguments, the arguments are not checked for data type.
  - #define show(x) (printf("\n value of x=%d", x)
  - #undef-used to delete the macro definition

Macro Definition

```
#define PI 3.14
main( )
{
float r = 1.50;
float area ;
area = PI * r * r ;
printf ( "\nArea of circle = %f",
area ) ;
}
```

Macro with Arguments

```
#define Area(x) ( 3.14 * x * x )
main( )
{
float r = 1.50; a ;
a = Area ( r ) ;
printf ( "\nArea of circle = %f", a ) ;
}
```

Preprocessor Tasks

3. Removing comments

- It removes all the comments.
- A comment is written only for the humans to understand the code and they are of no use to a machine.
- preprocessor removes all of them as they are not required in the execution and won't be executed as well.

Preprocessor Tasks

4. Conditional Compilation

- Compiler can skip some part of a source code if we want.

- preprocessing commands **#ifdef** and **#endif** are used for this purpose.

```
#ifdef macroname
        statement 1 ;
        statement 2 ;
        statement 3 ;
#endif
```

- If macro name has been #defined, the block of code will be processed otherwise compiler will skip this block of code.

Preprocessor Tasks

#if and #elif directive

- The #if directive can be used to test whether an expression evaluates to a nonzero value or not.
- If the result of the expression is nonzero, then subsequent lines upto a #else, #elif or #endif are compiled.
- If the result of the expression is zero, then #if statements will be skipped.

Preprocessor Tasks

Example:

```
main( )
{
        #if age >=30
                statement 1 ;
                statement 2 ;
        #else
                statement 3 ;
                statement 4 ;
        #endif
}
```

Command Line Argument

- In C programs, values can be passed from the command line when they are executed.
- These values are called command line arguments.
- handled using main() function arguments.
- int main(int argc, char *argv[])
- argc refers to the number of arguments passed, and argv[] is a pointer array which points to each argument passed to the program.

Command Line Argument

```
int main(int arg, char *argv[])
{
    int i,x,f=1;
    x=atoi(argv[1]);    //atoi() is used to convert string argument
value into integer value
   for(i=1;i<=x;i++)
    {
      f=f *i ;
    }
    printf("\n factorial=%d", f);
   return 0;
}
```

Types of Errors in C

There are five different types of errors in C.

- Syntax Error
- Run Time Error
- Logical Error
- Semantic Error
- Linker Error

1. Syntax Error

- Syntax errors occur when a programmer makes mistakes in typing the code's syntax correctly or makes typos.
- In other words, syntax errors occur when a programmer does not follow the set of rules defined for the syntax of C language.

- Syntax errors are sometimes also called compilation errors because they are always detected by the compiler.

- The most commonly occurring syntax errors in C language are:

  Missing semi-colon (;)
  Missing parenthesis ({})
  Assigning value to a variable without declaring it

2. Runtime Error

Errors that occur during the execution (or running) of a program are called RunTime Errors. These errors occur after the program has been compiled successfully.

For example, while a certain program is running, if it encounters the square root of -1 in the code, the program will not be able to generate an output because calculating the square root of -1 is not possible. Hence, the program will produce an error.

Runtime errors can be a little tricky to identify because the compiler can not detect these errors. They can only be identified once the program is running. *Some of the most common run time errors are: number not divisible by zero, array index out of bounds, string index out of bounds, etc*.

Runtime errors can occur because of various reasons. Some of the reasons are:

Mistakes in the Code: During the execution of a while loop, the programmer forgets to enter a break statement. This will lead the program to run infinite times, hence resulting in a run time error.

Memory Leaks: If a programmer creates an array in the heap but forgets to delete the array's data, the program might start leaking memory, resulting in a run time error.

Mathematically Incorrect Operations: Dividing a number by zero, or calculating the square root of -1 will also result in a run time error.

Undefined Variables: If a programmer forgets to define a variable in the code, the program will generate a run time error.

3. Logical Error

Sometimes, we do not get the output we expected after the compilation and execution of a program.

Even though the code seems error free, the output generated is different from the expected one. These types of errors are called Logical Errors.

Logical errors are those errors in which we think that our code is correct, the code compiles without any error and gives no error while it is running, but the output we get is different from the output we expected.

In 1999, NASA lost a spacecraft due to a logical error. This happened because of some miscalculations between the English and the American Units. The software was coded to work for one system but was used with the other.

4. Semantic Error

Errors that occur because the compiler is unable to understand the written code are called Semantic Errors.

A semantic error will be generated if the code makes no sense to the compiler, even though it is syntactically correct. It is like using the wrong word in the wrong place in the English language. For example, adding a string to an integer will generate a semantic error.

Semantic errors are different from syntax errors, as syntax errors signify that the structure of a program is incorrect without considering its meaning. On the other hand, semantic errors signify the incorrect implementation of a program by considering the meaning of the program.

The most commonly occurring semantic errors are: use of un-initialized variables, type compatibility, and array index out of bounds.

5. Linker Error

Linker is a program that takes the object files generated by the compiler and combines them into a single executable file.

Linker errors are the errors encountered when the executable file of the code can not be generated even though the code gets compiled successfully. This error is generated when a different object file is unable to link with the main object file.

We can run into a linked error if we have imported an incorrect header file in the code, we have a wrong function declaration, etc.

Thank you